

Week 1: Introduction to Mobile Applications using Flutter

Dr M AOUE
Lecture notes

Objectives. By the end of this lecture you will know what this course covers, what Dart and Flutter are and why we use them, what you need to install, why we use Git and GitHub, and how to write, run, and compile your first Dart programs (variables, operators, control flow, and console input).

1 Why are we taking this course?

This course is an introduction to creating mobile applications.

- We will learn how to create a mobile GUI application and deploy it to emulators and mobile devices.
- You will learn how to connect your applications to a remote database through RESTful services.
- You will learn these concepts through the **Flutter** framework.

2 Course Topics

1. Dart programming language basics.
2. Creating mobile GUI applications using Flutter.
3. Creating online databases.
4. How to create and access REST services.
5. Publishing your projects using a source control system (Git).

3 What is Dart?

- Flutter uses the **Dart** programming language.
- Dart was originally designed to create web applications, and it was originally compiled to JavaScript.
- Today Dart can be compiled to many platforms, such as Android, iOS, Windows, and Linux.

4 How to use Dart?

There are several ways to write and run Dart code.

Option	Description
DartPad	An online service to write and test Dart code: https://dartpad.dev
Flutter SDK	Download it from https://flutter.dev ; it already includes the Dart SDK.
Replit	A cloud-based platform: https://replit.com
Visual Studio Code	Optional; useful for writing Dart code locally on your computer: https://code.visualstudio.com

5 What is Flutter?

Flutter is a framework to deploy applications from a **single source code** to a variety of platforms.

Examples	
You can deploy to	Android, iOS, Windows, Linux, Web
You can write code with	Android Studio, VS Code

Advantages of Flutter

- Can be deployed to multiple platforms.
- Compiles to native machine code, so it is fast and efficient.
- Has a **hot reload** feature, which speeds up testing and deploying your applications.
- Fast rendering of GUI components: it allows partial refresh of screen components, which makes screen rendering faster.

6 Installing and Configuring Flutter

- The minimum requirements are 8 GB of RAM and Windows 10 or above.
- Install the **Flutter SDK**, which includes the Dart SDK.
- Install **Git**, since Flutter relies heavily on it.
- Configure the Flutter SDK: <https://docs.flutter.dev/install>
- Since we will develop applications for Android, install Android Studio and the Android Emulator: <https://developer.android.com/studio>

Note. You do not write Java or Kotlin in this course: your code is Dart. Android Studio still installs the Android SDK and emulator that Flutter needs to build and run Android apps.

Step-by-step guides: 001_install (Android Studio edition) and 001_install_vscode (VS Code edition). Use whichever editor you prefer.

7 Source Version Control Platforms

Before source version control platforms existed, a programmer working on a project would:

1. Create a directory for the project.
2. Write some working code so that the project implements some features.
3. Back up the project before adding more features.

These tasks become harder when we work in a group with other programmers. A source version control platform automates them and allows multiple programmers to work on the same code base.

8 Git and GitHub

- Git is one of the most widely used source version control platforms.
- Git and GitHub are **not** the same. Git contains the tools for version control, while GitHub is an online cloud system where programmers store their source code.
- You can use Git with other hosting platforms such as BitBucket and GitLab.
- Learn Git: <https://www.w3schools.com/git>
- Download Git: <https://git-scm.com/downloads>

Your projects must be published on GitHub.

9 Writing your first Dart program

- Go to <https://dartpad.dev> and write your first “hello world” program.
- Dart is both a procedural and an object-oriented (OOP) language. Declaring classes is therefore optional.
- Your program must have at least one function called **main**. It is the starting point of your program.
- DartPad translates Dart code to JavaScript to run it inside a web browser.
- DartPad cannot take user console input. You need the Dart SDK for that.
- You can also use Replit, which has a pre-installed Dart SDK running on Linux.

```
1 void main() {
2   print('hello world');
3 }
```

Listing 1: hello world

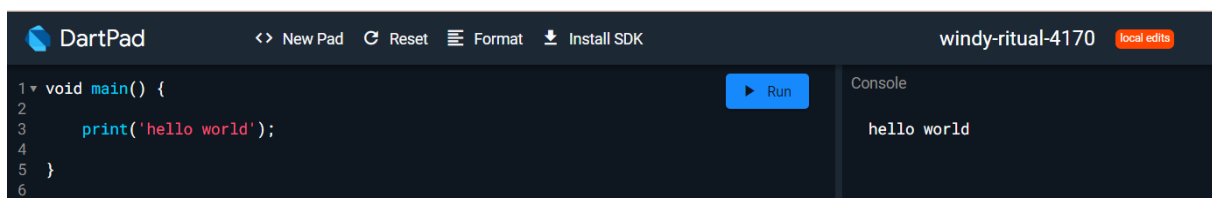


Figure 1: The program in DartPad, with its output in the console.

10 Running Dart programs using the SDK

You can write Dart programs with your favorite text editor, such as Notepad, save the file, and execute it with the **dart** command. The example below assumes the code was saved in a file called **test.dart** inside the **dart_programs** folder.

```
C:\Users> cd c:\dart_programs

c:\dart_programs> dart test.dart
hello world
```

11 Running Dart programs using VS Code

You can write Dart programs with Visual Studio Code and the Dart plugin, as shown below.

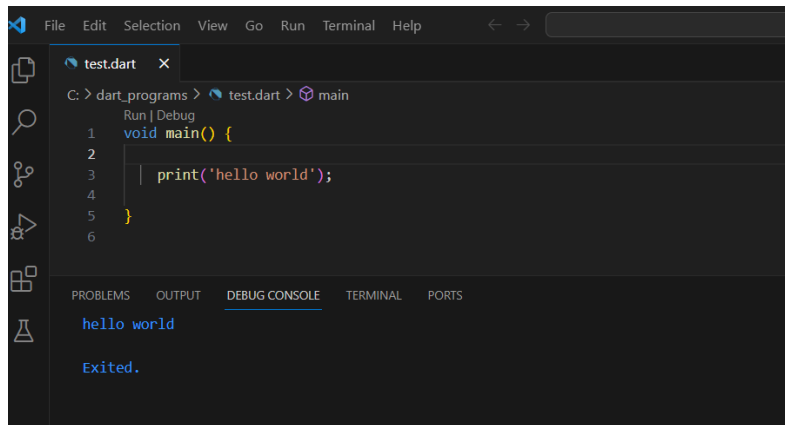


Figure 2: `test.dart` in VS Code; the output appears in the Debug Console.

12 Compile your program to machine code

You can compile your code to an executable `.exe` file on Windows, then run it without using the `dart` command again.

```
c:\dart_programs> dart compile exe -o test.exe test.dart
Generated: c:\dart_programs\test.exe

c:\dart_programs> test
hello world
```

- The compiled program is faster than running it with the `dart` command.
- You must re-compile your program each time you change the source code.

13 Adding 2 numbers

Dart is a strongly typed language, although declaring variable types is optional. Dart infers a variable's type from its assigned value: in the left-hand code, `x` and `y` become integers. You can also declare the types explicitly for clarity, as in the right-hand code.

```
void main() {
  var x = 10;
  var y = 20;
  var sum = x + y;
  // print adds a new line
  // after the printed string
  print('Result: $sum');
}
```

```
void main() {
  int x = 10;
  int y = 20;
  int sum = x + y;
  // $ makes Dart use the variable
  // value, converted to a string
  print('Result: $sum');
}
```

14 Dart is a strongly typed language

Once a variable's type is inferred, you cannot assign values of a different type to it. In the code below, because we assigned an integer to `x`, Dart defines its type as `int`. Assigning a string afterwards generates a **compile-time error**.

```
1 void main() {
2   var x = 10;
3   x = 'some text'; // error: a String can't be assigned to an int
4   print('Value: $x');
5 }
```

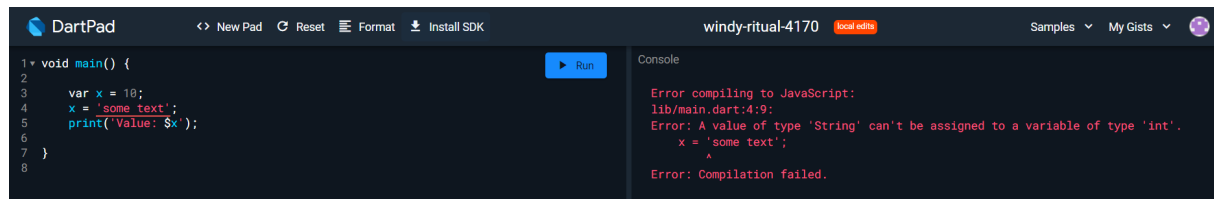


Figure 3: DartPad refuses to compile the program above.

15 Dart Arithmetic operators

Dart has the typical operators found in many languages.

Operator	Meaning
+	Addition of numbers. It can also concatenate strings.
-	Subtraction.
*	Multiplication.
/	Division. A simple division with <code>/</code> always results in a <code>double</code> value.
<code>~/</code>	Integer division. In many languages you would need a type cast to get only the integer part; in Dart this operator does it.
<code>%</code>	Modulo (the remainder of integer division).
<code>-expression</code>	Negation (reverses the sign of the expression).

16 Equality and relational operators

Operator	Checks whether
<code>==</code>	the operands are equal
<code>!=</code>	the operands are different
<code>></code>	the left operand is greater than the right one
<code><</code>	the left operand is less than the right one
<code>>=</code>	the left operand is greater than or equal to the right one
<code><=</code>	the left operand is less than or equal to the right one

Note. In Dart, unlike Java, `==` does not compare memory references for numbers and strings: it compares their content (the values).

17 Dart Data Types

Type / keyword	Description
final, const	The value of the variable cannot be changed once it is initialized.
int	64-bit signed non-fractional integer values, from -2^{63} to $2^{63} - 1$.
double	Fractional numeric values, as a 64-bit double-precision floating-point number.
bool	Its value is either true or false .
BigInt	Arbitrary-precision integers: the size limit is the RAM of the computer running the program. Useful depending on the context, but slower than the num types, so consider this when choosing it.
String	A sequence of characters (UTF-16 code units), mainly used to represent text.

18 Control flows and looping

We have seen how to use Dart variables and operators to create conditional expressions. To make our code take the right direction in our logic, we need control flow. Dart's control flow syntax is very similar to other languages:

- **if-else**
- **switch/case**
- Looping with **for**, **while**, and **do-while**
- **break** and **continue**

These are used like in Java, so we will not go over them in detail.

19 Example: printing the even numbers between m and n

Notice the use of **\$** to print a variable's value inside the quotes.

```

1 void main() {
2   int m = 10;
3   int n = 20;
4
5   for (int i = m; i <= n; i++) {
6     if (i % 2 == 0) {
7       print("$i");
8     }
9   }
10 }
```

Output:

```

10
12
14
16
18
20
```

20 Console Input

DartPad compiles source code to JavaScript, so it cannot take console input. We need the Dart SDK. The example below was written in Visual Studio Code.

Dart is **null safe**: when we declare the `String` variables we add a `?` to the type, because `first` and `last` may have a null value.

```
1 import 'dart:io';
2
3 // The package above is required for the write and readLineSync functions
4 void main() {
5     String? first, last; // we need ? because its value may be null
6     stdout.write('Enter first name: '); // write does not add a new line
7     first = stdin.readLineSync(); // this waits for user input
8     stdout.write('Enter last name: ');
9     last = stdin.readLineSync();
10    print('Welcome $first $last');
11 }
```

21 Taking Integer Input

- `readLineSync` returns a string, so to take integer input you convert it with the `int.parse` function.
- `readLineSync` may return null, so we use the null assertion operator (`!`) to make Dart treat it as non-nullable.

```
1 import 'dart:io';
2
3 // The package above is required for the write and readLineSync functions
4 void main() {
5     int x, y;
6     stdout.write('Enter x: ');
7     x = int.parse(stdin.readLineSync()!);
8     stdout.write('Enter y: ');
9     y = int.parse(stdin.readLineSync()!);
10    print('Sum is ${x + y}');
11 }
```

This document is maintained by Dr. Mohamad AOUDÉ

Last updated: 2026-09-20